News Finder Service with MongoDB Atlas

Estimated time needed: **45** minutes

Objectives

After completing this lab, you will be able to:

- Generate vector embeddings for NEWS data using a TensorFlow model
- Connect to the MongoDB Atlas database and create a table with Vector Embeddings
- Write an endpoint using Express server that takes a sentence as input and returns any news similar to the conviction entered.

Running the lab

Currently, the Skills Network Cloud IDE, which utilizes Theia and Docker technologies, does not support this lab. You have two options to participate in this **optional** lab.

1. Use MongoDB Atlas on MongoDB Cloud as a fully managed service by following steps in this lab.
2. Optionally, you can install MongoDB Atlas locally on your computer using Docker by following the steps in this lab.

This lab assumes that you have picked option 1 above.

Prerequisites

MongoDB Atlas on the Cloud

Ensure you have created an account on MongoDB Cloud and provisioned an Atlas database by following steps in this lab.

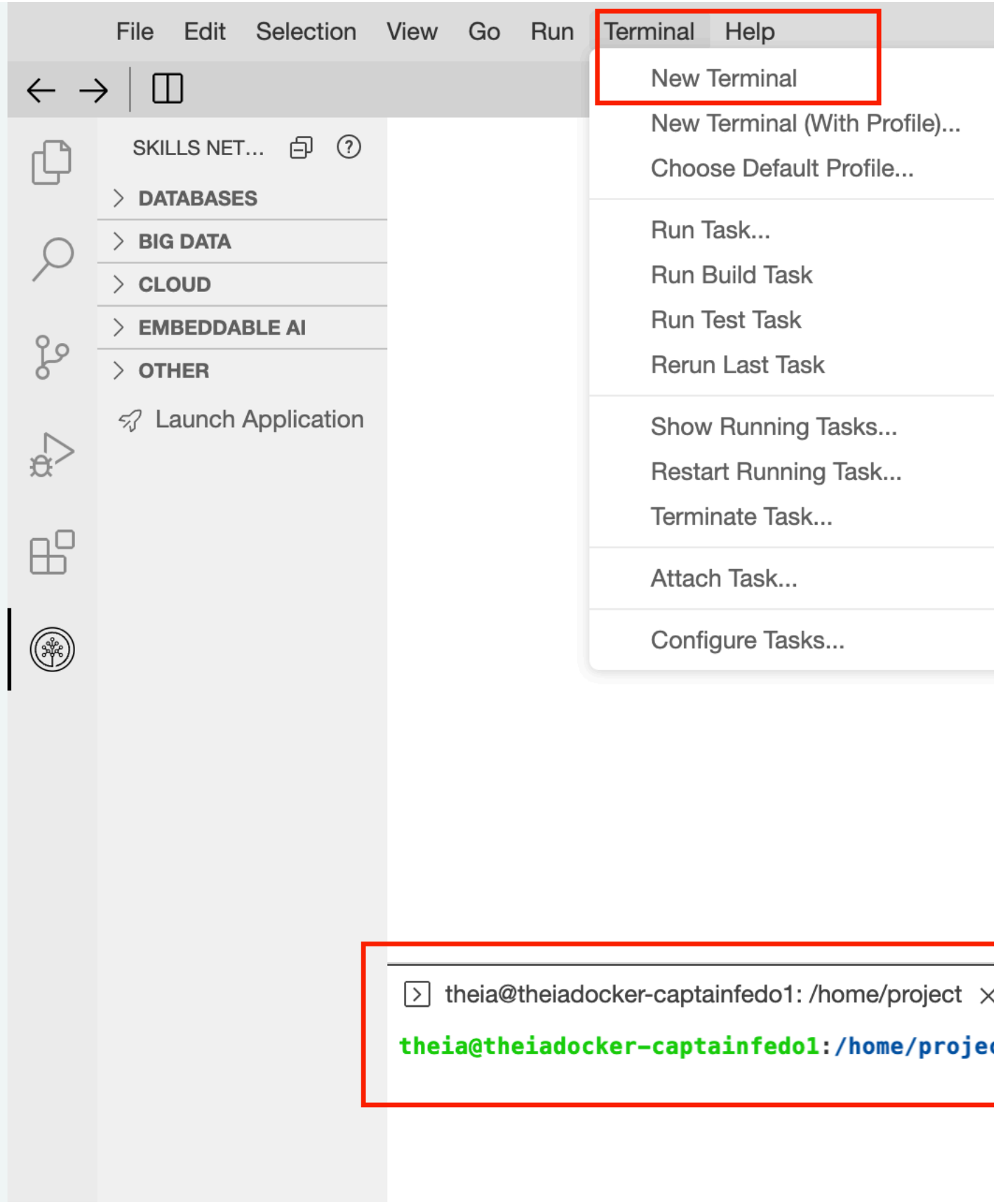
Create directories

In this lab, you will work with MongoDB Atlas, **not** the regular MongoDB database. The Atlas version simplifies the vector storage, indexing, and searching processes, and you will use all the features in this lab. You will search for similar news items using the `cosine` function. Here is a list of all the similarity functions available, along with their descriptions:

- `cosine`: Measures the cosine of the angle between two vectors. A higher cosine value indicates more remarkable similarity.
- `euclidean`: Measures the straight-line distance between two points (vectors) in a multidimensional space. A lower distance indicates higher similarity.
- `dotproduct` calculates the sum of the product of corresponding components between two vectors. Higher dot product values indicate greater alignment.

  Note: You can learn more about it in the MongoDB Atlas vector search documentation.

1. Open a terminal in the IDE by using the `Terminal` -> `New Terminal` menu. You should see the terminal appear at the bottom of the screen.



2. Let's create a directory for the project and change it into that directory.

```
mkdir mongodb-vector && cd mongodb-vector
```

3. Let's also save the full path of the folder in an environment variable that you can refer to later.

```
export VECTOR_HOME=$(pwd)
```

4. You can use the `echo` command to see what's inside this variable.

```
echo $VECTOR_HOME
```

The output of this command will show the value of the `VECTOR_HOME` environment variable. If this is empty, rerun the previous command.

```
/home/project/mongodb-vector
```

5. Let's create a data sub-directory using this variable. Adding a subdirectory is a good practice that ensures better organization and management of files.

```
mkdir $VECTOR_HOME/data
```

Seed data

Before writing any code, let's view the data file. The JSON file containing an array of headlines provides the seed data. Each headline in the file has the following attributes:

- **Link:** URL of the headline
- **Headline:** Text of the headline
- **Category:** Type of headline (world news, entertainment, politics, and so on)
- **Short_description:** More about the headline
- **Authors:** Author of the headline
- **Date:** The date of posting the headline in YYYY-MM-DD format


```
[
    {
        "link": "https://www.huffpost.com/entry/newspaper-front-pages-mark-the-queens-death_n_631b09e1e4b0eac9f4d64686",
        "headline": "'Our Hearts Are Broken': Historic Front Pages Mark The Queen's Death",
        "category": "WORLD NEWS",
        "short_description": "Both British and international newspapers honor the passing of the UK's longest-reigning monarch.",
        "authors": "Kate Nicholson",
        "date": "2022-09-09"
    },
    {
        "link": "https://www.huffpost.com/entry/jojo-siwa-glsen-gamechanger-award_n_631bae79e4b082746bdfc964",
        "headline": "JoJo Siwa To Receive Prestigious GLSEN Honor For Her LGBTQ Advocacy Work",
        "category": "ENTERTAINMENT",
```

```
    "short_description": "The 19-year-old singer, dancer, and YouTube icon will receive the 2022 Gamechanger Award for her focus on anti-bullying
    "authors": "",
    "date": "2022-09-09"
  }
]
```


Data attribution

You can get the complete file using the following `wget` or `curl` command or by opening the URL for the JSON file in your browser and saving the file locally.

WGET

```
wget -O $VECTOR_HOME/data/newsData.json https://cf-courses-data.s3.us.cloud-object-storage.appdomain.cloud/T4GkekRs6tCVb3vElgmQJQ/newsData.json
```

CURL

```
curl -o $VECTOR_HOME/data/newsData.json https://cf-courses-data.s3.us.cloud-object-storage.appdomain.cloud/T4GkekRs6tCVb3vElgmQJQ/newsData.json
```

Task 1: Create Node.js project and install dependencies

Let's create the node package and install all the dependencies.

1. Ensure you are still in the $VECTOR_HOME directory and run `npm init -y` command to create a node package.

   ```
   npm init -y
   ```

   The output should look like:

   ```
   ➜  mongodb-vector npm init -y
   Wrote to /home/project/mongodb-vector/package.json:
   {
     "name": "mongodb-vector",
     "version": "1.0.0",
     "description": "",
     "main": "index.js",
     "scripts": {
       "test": "echo \"Error: no test specified\" && exit 1"
     },
     "keywords": [],
     "author": "",
     "license": "ISC"
   }
   ```

2. Install the required packages, including `@tensorflow-models/universal-sentence-encoder`, `@tensorflow/tfjs-node`, `dotenv`, `mongodb`, and `express`.

    - @tensorflow-models/universal-sentence-encoder: The universal sentence encoder model
    - @tensorflow/tfjs-node: JavaScript version of TensorFlow
    - dotenv: Loads environment variables from a .env file into process.env
    - mongodb: A library to work with mongoDB
    - express: Web framework for Node.js

    ```
    npm install @tensorflow-models/universal-sentence-encoder @tensorflow/tfjs-node dotenv express mongodb
    ```

    The output is long but should end with libraries installed:

    ```
    added 197 packages and audited 198 packages in 42s
    21 packages are looking for funding
      run `npm fund` for details
    2 moderate severity vulnerabilities
    To address all issues, run:
      npm audit fix
    Run `npm audit` for details.
    ```

You can safely ignore the warnings for now.

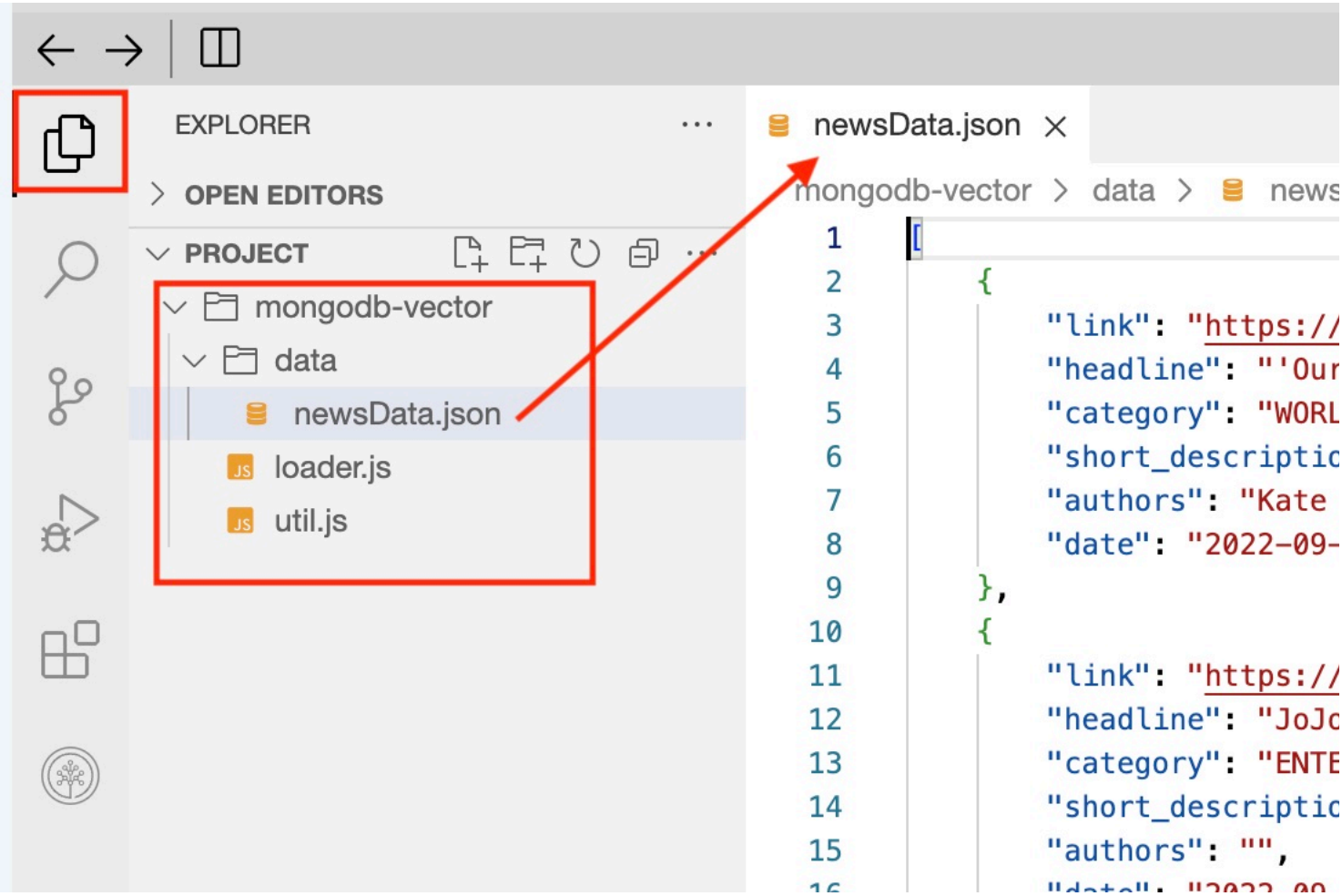
Task 2: Create common util functions

Create util.js file

This file will contain the common reusable code

```
touch util.js
```

The next step of commands will ask you to add code to some files. You can open the specified file in the Cloud IDE by using the file explorer panel. If doing this lab locally, feel free to use your favourite code editor.



Import statements

We import a few libraries at the top of the file:

- require ('dotenv') .config ();- imports variables from the .env file. This file has the `MONGO_HOST`, `MONGO_PORT`, `MONGO_USER`, `MONGO_PASS`, `MONGO_DB`, and `MONGO_COLLECTION` constants defined. Normally, you would not share this file in the GitHub repo or with anybody else, but we are sharing it here to make the exercise more straightforward. Feel free to change the variables in this file.
- `const { MongoClient } = require('mongodb');` - imports the mongodb npm package. This package is used to interact with MongoDB from a JavaScript program.
- `require("@tensorflow/tfjs-node");` - used in Node.js applications to include TensorFlow.js for Node.js, a JavaScript library for training and deploying machine learning models in Node.js.

Create MongoDB URL from the properties file

The next few lines read to constants from the `.env`file and set local variables for the rest of the code, like setting up the MongoDB connection string.

```
require('dotenv').config();
const { MongoClient, ServerApiVersion } = require('mongodb');
require("@tensorflow/tfjs-node");
const us_encoder = require("@tensorflow-models/universal-sentence-encoder");
const { MONGO_HOST, MONGO_PORT, MONGO_USER, MONGO_PASS, MONGO_DB , MONGO_COLLECTION } = process.env;
const uri = `mongodb+srv://${MONGO_USER}:${MONGO_PASS}@${MONGO_HOST}/?retryWrites=true&w=majority`;
```

Create a vector from a sentence

The `vectorizeText` function takes text as input and creates an embedding or a vector using the `universal-sentence-encoder` model from TensorFlow. We are also using the `loadModel()` method to ensure the model is only downloaded once, thus improving performance.

```
let client;
let model;
async function loadModel() {
  if (!model) {
    model = await us_encoder.load();
```

```
  }
}
async function vectorizeText(text) {
  await loadModel();
  const embeddingsRequest = await model.embed(text);
  const vector = embeddingsRequest.arraySync()[0];
  return vector;
}
```

Connect to MongoDB

The file also has a util function to connect to MongoDB using the `MongoClient`. The function returns a promise, so we use the `await` keyword.

```
async function connectToMongoDB() {
    if (!client) {
        client = new MongoClient(uri);
        try {
            await client.connect();
            console.log('Connected to MongoDB Atlas');
        } catch (err) {
            console.error('Error connecting to MongoDB:', err);
            throw err;
        }
    }
    return client.db(MONGO_DB);
}
```

Close the MongoDB connection

Finally, the `closeMongoDBConnection` method closes the connection and sets the client to null.

```
async function closeMongoDBConnection() {
    if (client) {
        await client.close();
        console.log('MongoDB connection closed');
        client = null;
    }
}
```

Finally, the file exports all the functions and the `MONGO_COLLECTION` so it can be used elsewhere.

```
module.exports = { vectorizeText, connectToMongoDB, closeMongoDBConnection, MONGO_COLLECTION };
```

Solution

Here is the complete file if you are stuck

▶ Click here for the util.js

Task 3: Load the vectors into MongoDB Atlas

Initial data

You downloaded the JSON file with the complete data earlier. Each news article has a `link`, `headline`, `category`, `short description`, `authors`, and `date` attribute. This exercise aims to read this JSON file into memory and then create vector embeddings for each `short_description` and store them in the database along with the rest of the data. This will look like the following in the database:


```
{
  "_id": {
    "$oid": "6612e83af5d8ed35e5a7b83c"
  },
  "link": "https://www.huffpost.com/entry/newspaper-front-pages-mark-the-queens-death_n_631b09e1e4b0eac9f4d64686",
  "headline": "'Our Hearts Are Broken': Historic Front Pages Mark The Queen's Death",
  "category": "WORLD NEWS",
  "short_description": "Both British and international newspapers honor the passing of the UK's longest-reigning monarch.",
  "authors": "Kate Nicholson",
  "date": "2022-09-09",
  "description_vector": [
    -0.05890331044793129,
    -0.036208368837833405,
    0.05714097619056702,
    0.02889532968401909,
    -0.025822531431913376,
    -0.004176020622253418,
    0.008981405757367611,
    ...
    ...
    0.0125401820987463
    -0.04702383279800415
  ]
}
```


The `description_vector` is an array of 512 decimal numbers that describe the short description of the news article.

Set up configuration

1. Create a file called `.env` in the $VECTOR_HOME directory.

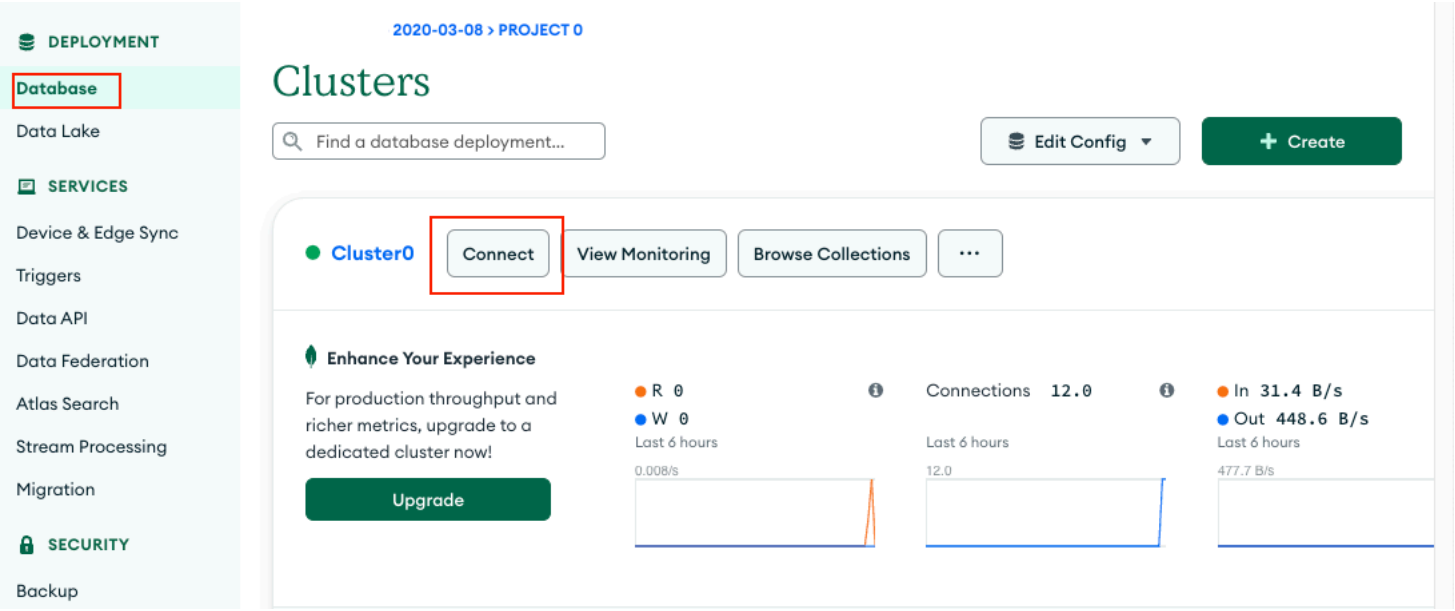
```
touch .env
```

Populate the file with the following information:

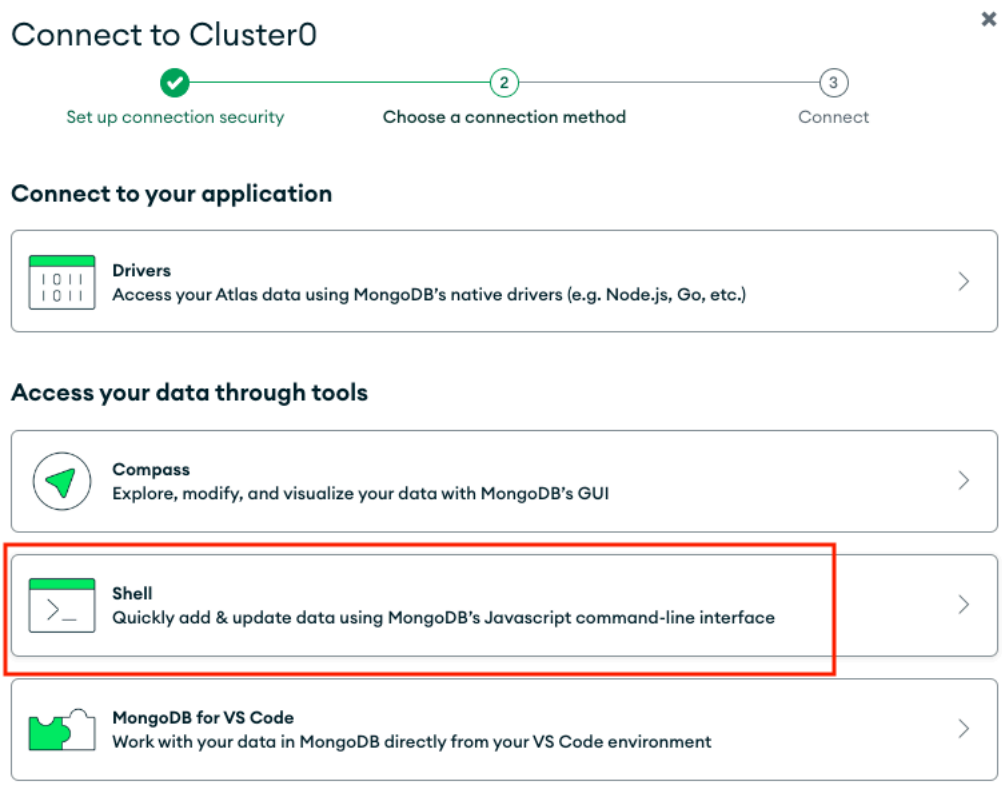
```
MONGO_HOST=localhost
MONGO_PORT=27017
MONGO_USER=root
MONGO_PASS=root
MONGO_DB=myvectordb
MONGO_COLLECTION=vectorized_data
```

Note: Do not use `MONGO_PASSWORD` as the key name as this is reserved in the Cloud IDE.

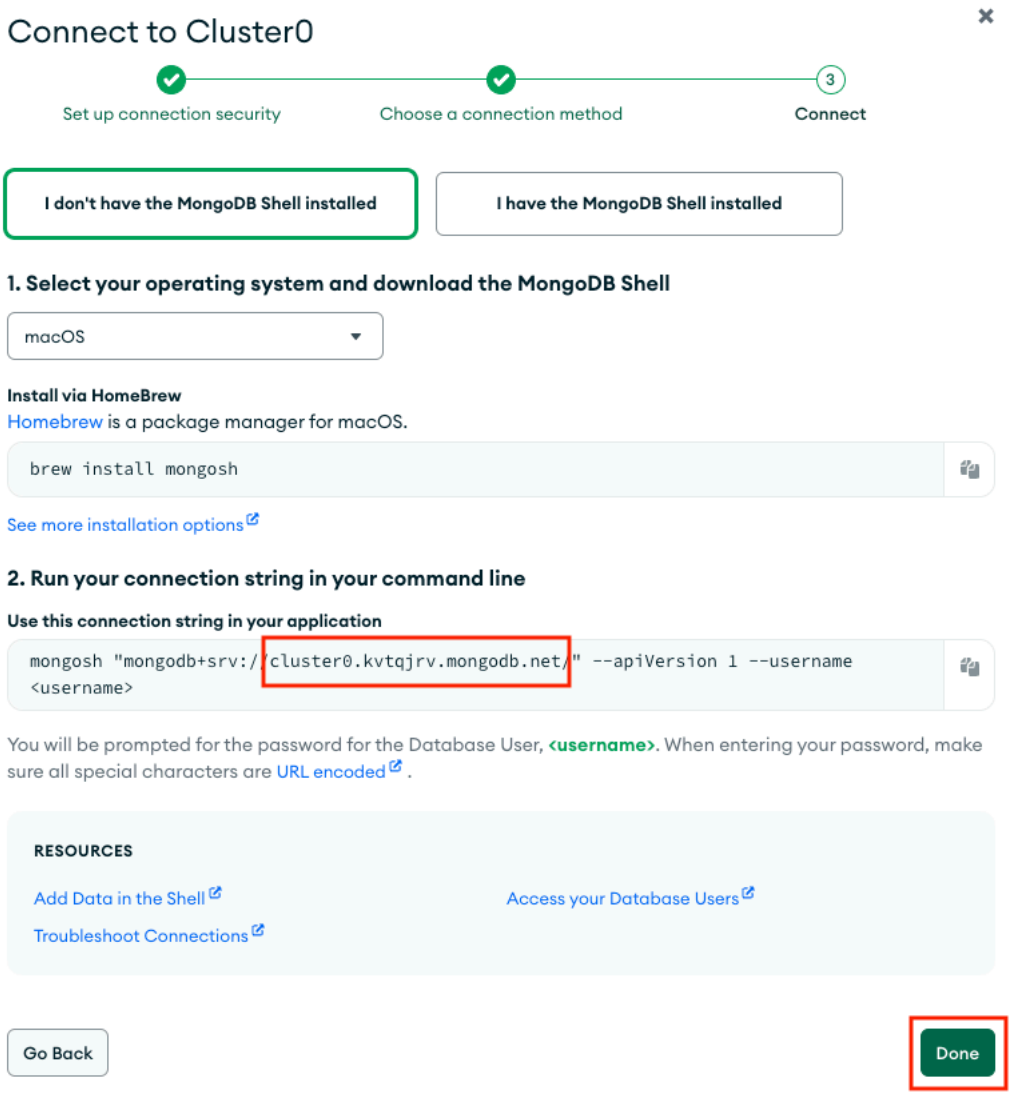
Obtain the host
You can get the hostname by opening the `Connect` dialog from your database. Open the dialog:



You can pick any method to connect to see your host name. This example shows the `Shell` option.

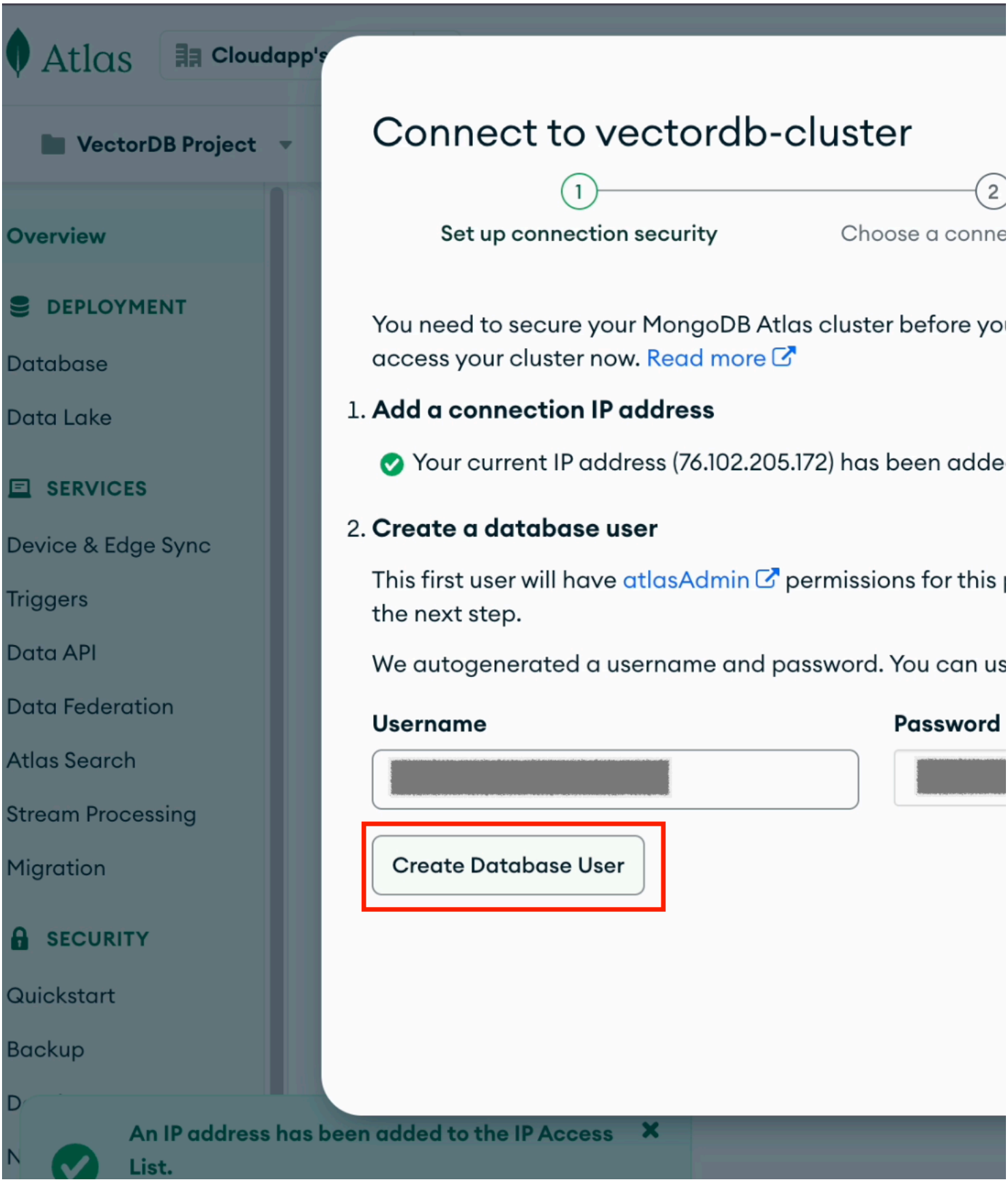


You can copy the host from the URI string and click `Done` to close the dialog. You just need the host name from the complete URI.

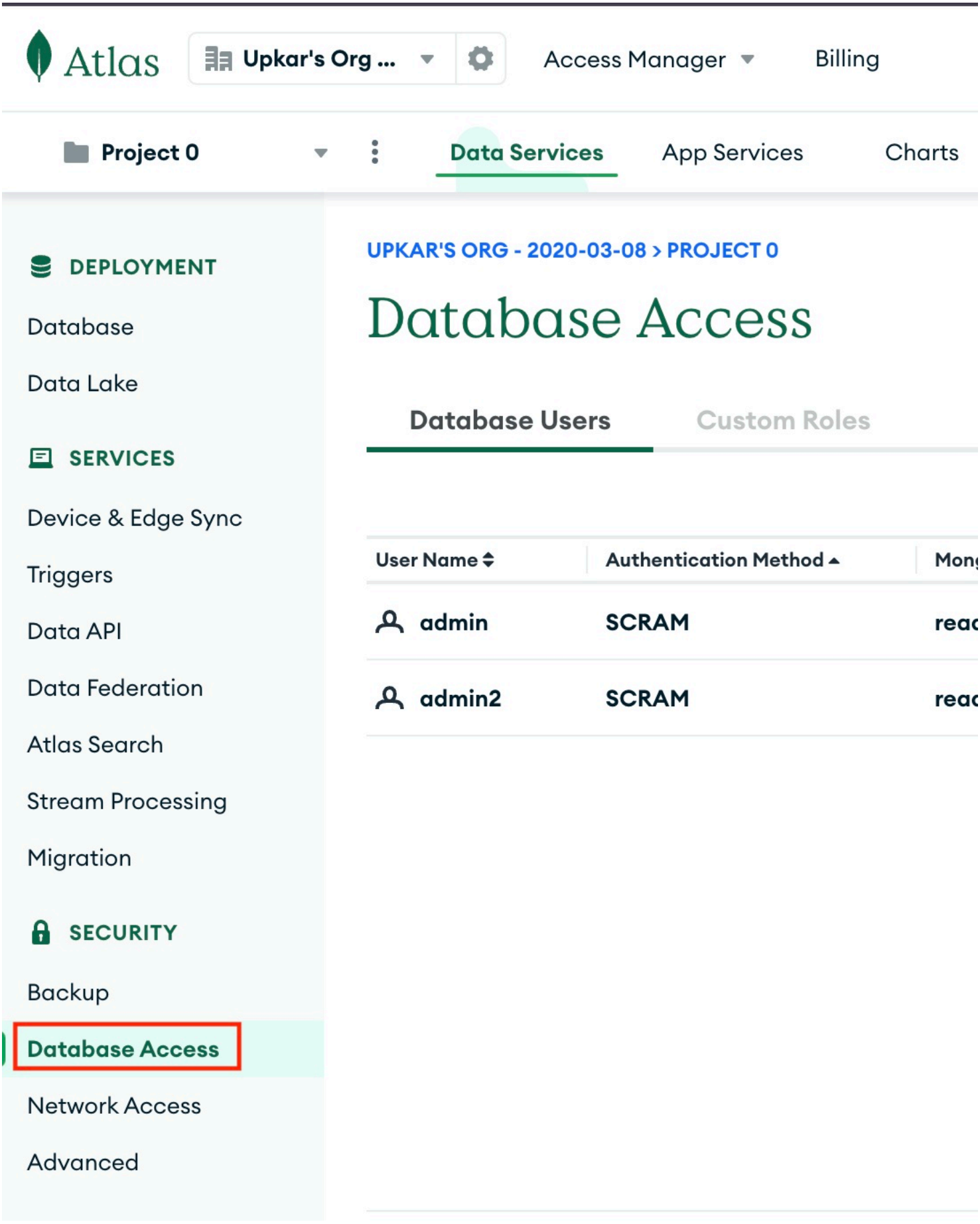


Obtain username and password

You created the username and password earlier when you created a user for your cluster. Recall this screen:



If you did not copy the username and password earlier, you can simply create a new user by logging into MongoDB Cloud and going to the `Database Access` section.



File to upload vectorized data into the collection

Create a new file named `loader.js`.

```
touch loader.js
```

This file will contain the method that loads the initial news articles to the MongoDB database. Let's fill out the loader.js file one method at a time.

Import initial data and common util functions

First, we use the `require` keyword to import the news articles JSON file and the functions we wrote in the util.js file in **Task 2: Create Common Utils Functions**.

```
const newsArray = require('./newsData.json');
const { vectorizeText, connectToMongoDB, closeMongoDBConnection, MONGO_COLLECTION } = require('./util');
```

Create vector embeddings and load the data

Next, we connect to MongoDB and use the `vectorizeText` function from the util file to create the embedding for the short_description for each news article:

```
for (const newsItem of newsArray) {
    const text = newsItem.short_description;
    const description_vector = await vectorizeText(text);
}
```

Finally, we save the data to the database using the `collection.insertOne` method and close the database connection using the `await closeMongoDBConnection();` method.

Run the code

Run the file using the `node` command.

```
node loader.js
```

You should see the new articles load one by one in the output, as shown here as a sample:

```
➜  mongodb-vector node loader.js
Platform node has already been set. You are overwriting the platform with node.
mongodb://root:root@localhost:27017?directConnection=true
Connected to MongoDB Atlas
Vectorized data stored in MongoDB Atlas for: 'Our Hearts Are Broken': Historic Front Pages Mark The Queen's Death
Vectorized data stored in MongoDB Atlas for: JoJo Siwa To Receive Prestigious GLSEN Honor For Her LGBTQ Advocacy Work
Vectorized data stored in MongoDB Atlas for: UN Chief Asks World For 'Massive' Help In Flood-Hit Pakistan
Vectorized data stored in MongoDB Atlas for: Politician's DNA Connected To Las Vegas Journalists Murder, Police Say
Vectorized data stored in MongoDB Atlas for: Michelle Obama Celebrated For Wearing Braids To Her White House Portrait Unveiling
Vectorized data stored in MongoDB Atlas for: Norman Reedus Opens Up About 'Walking Dead' Injury: 'I Thought I Was Going To Die'
Vectorized data stored in MongoDB Atlas for: Michigan Supreme Court Revives Abortion Rights Amendment For November's Ballot
Vectorized data stored in MongoDB Atlas for: Portland Residents With Disabilities Sue Over City's Blocked Sidewalks
Vectorized data stored in MongoDB Atlas for: At Least 32 Dead In Fire At Karaoke Parlor In South Vietnam
Vectorized data stored in MongoDB Atlas for: Baseball Players Union Joins AFL-CIO In Show Of Solidarity With Other Workers
Vectorized data stored in MongoDB Atlas for: The Unemployment Insurance System Is Not Ready For The Next Recession
```

```
Vectorized data stored in MongoDB Atlas for: Power Outage At Austin Airport Leads To Flight Delays, Traffic Jams
Vectorized data stored in MongoDB Atlas for: Shelling Goes On Near Ukraine Nuclear Plant, Despite Risks
Vectorized data stored in MongoDB Atlas for: Cyberattack Prompts Los Angeles School District To Shut Down Its Computer Systems
Vectorized data stored in MongoDB Atlas for: Fast-Moving Fairview Fire Kills At Least 2 In California
Vectorized data stored in MongoDB Atlas for: Kody Clemens Strikes Out MVP Shohei Ohtani, Trails Dad Roger By 4,671 Ks
Vectorized data stored in MongoDB Atlas for: Mississippi Governor Says Water Pressure Is Now 'Solid' In Jackson
Vectorized data stored in MongoDB Atlas for: Meta, Parent Company Of Instagram, Fined $400 Million By Irish Regulators
Vectorized data stored in MongoDB Atlas for: School Starts Today For Survivors Of Deadly Mass Shooting In Uvalde
Vectorized data stored in MongoDB Atlas for: Man Accused Of Stuffing Grandmother In Freezer To Die
Vectorized data stored in MongoDB Atlas for: Banksy Went On 'Spraycation' And All We Got Was Some 'Mindless Vandalism'
Vectorized data stored in MongoDB Atlas for: Rob Lowe's Story About Steak With Chris Farley Is Well-Done
Vectorized data stored in MongoDB Atlas for: Gigi Hadid Denies Claim That She's Trying To 'Disguise' Her Pregnancy
Vectorized data stored in MongoDB Atlas for: Bonebreakers Contortionists Pull Off Mind-Blowing Act On 'America's Got Talent'
MongoDB connection closed
```

If you don't get an error, the articles were vectorized and loaded into the collection. If you run into a blocker, use the final solution shown here.

▶ Click here for the file

Task 4: Create an index based on the vector data

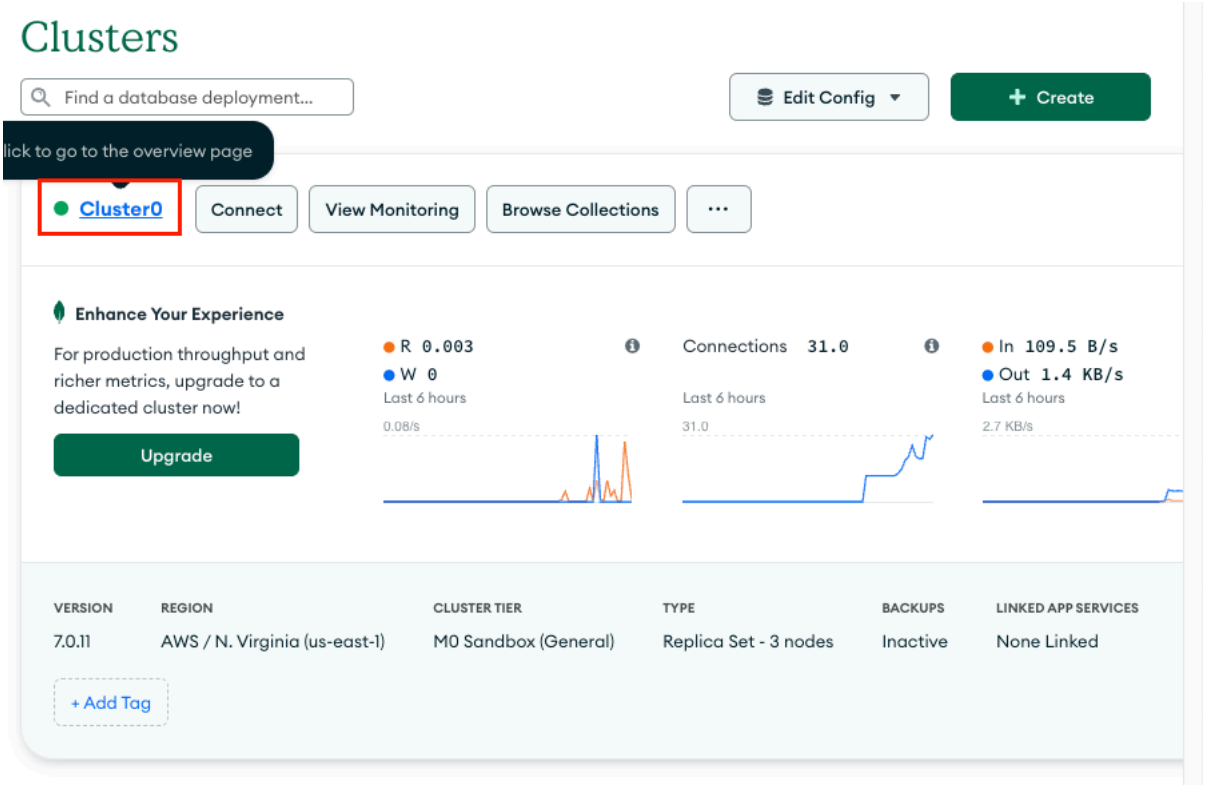
You will create an index over the vectorized data to enable searching using Atlas vector search.

Log into MongoDB Cloud

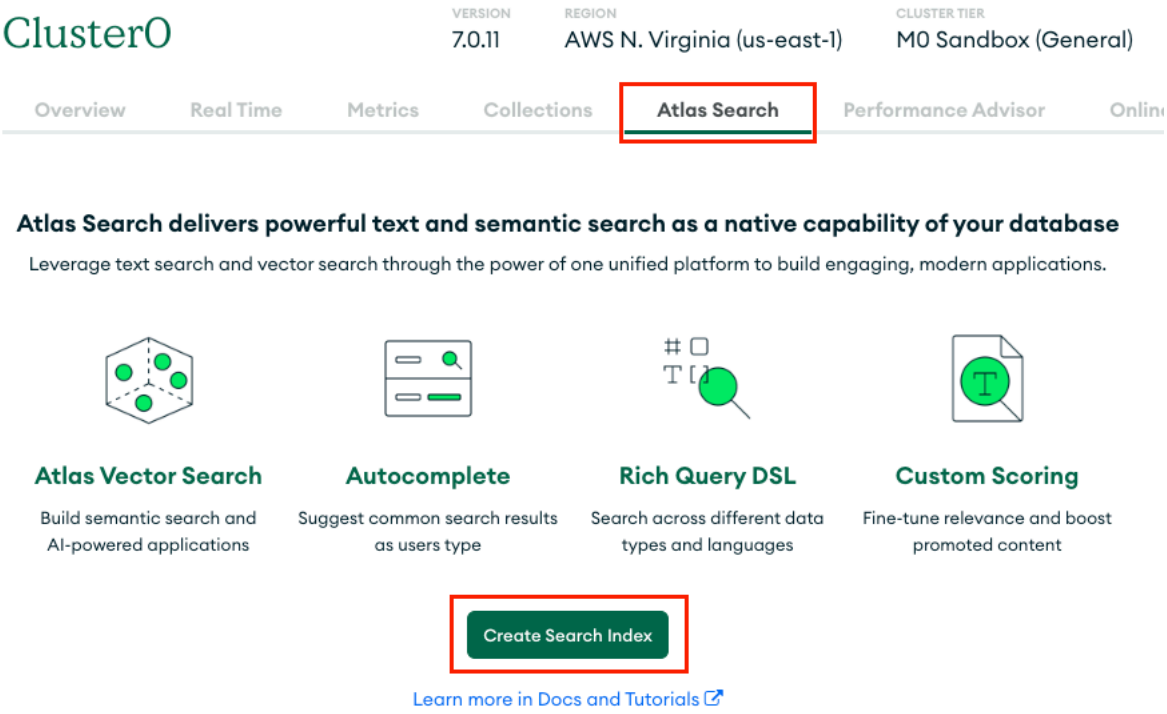
Log into MongoDB cloud if you haven't already or were logged out for some reason.

Create an index on cluster

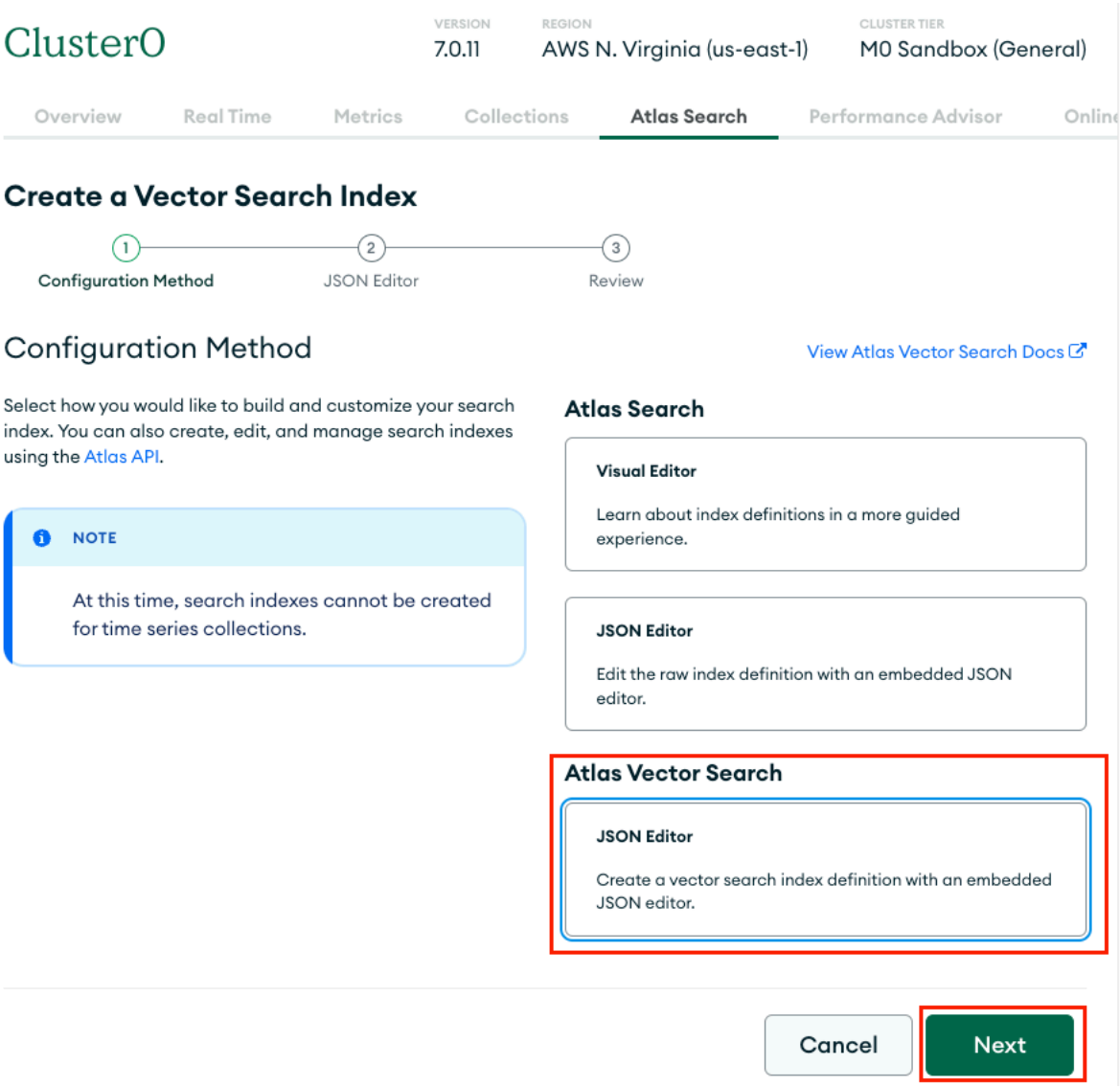
Click on your cluster to open the details page.



Click on `Atlas Search` tab and then `Create Search index`.



Pick `Atlas Vector Search` for the **Configuration Method**.

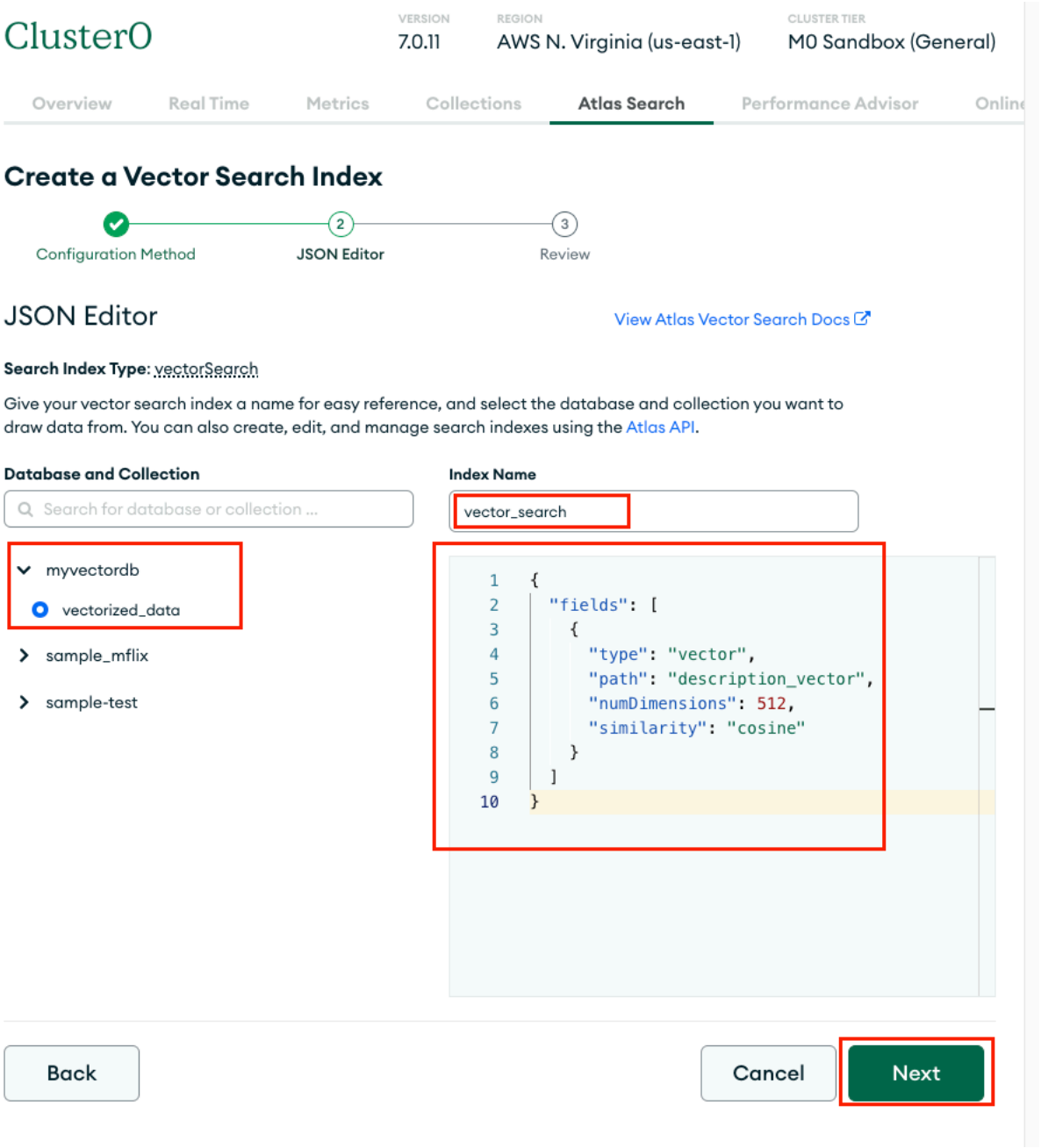


You have to do multiple things here.

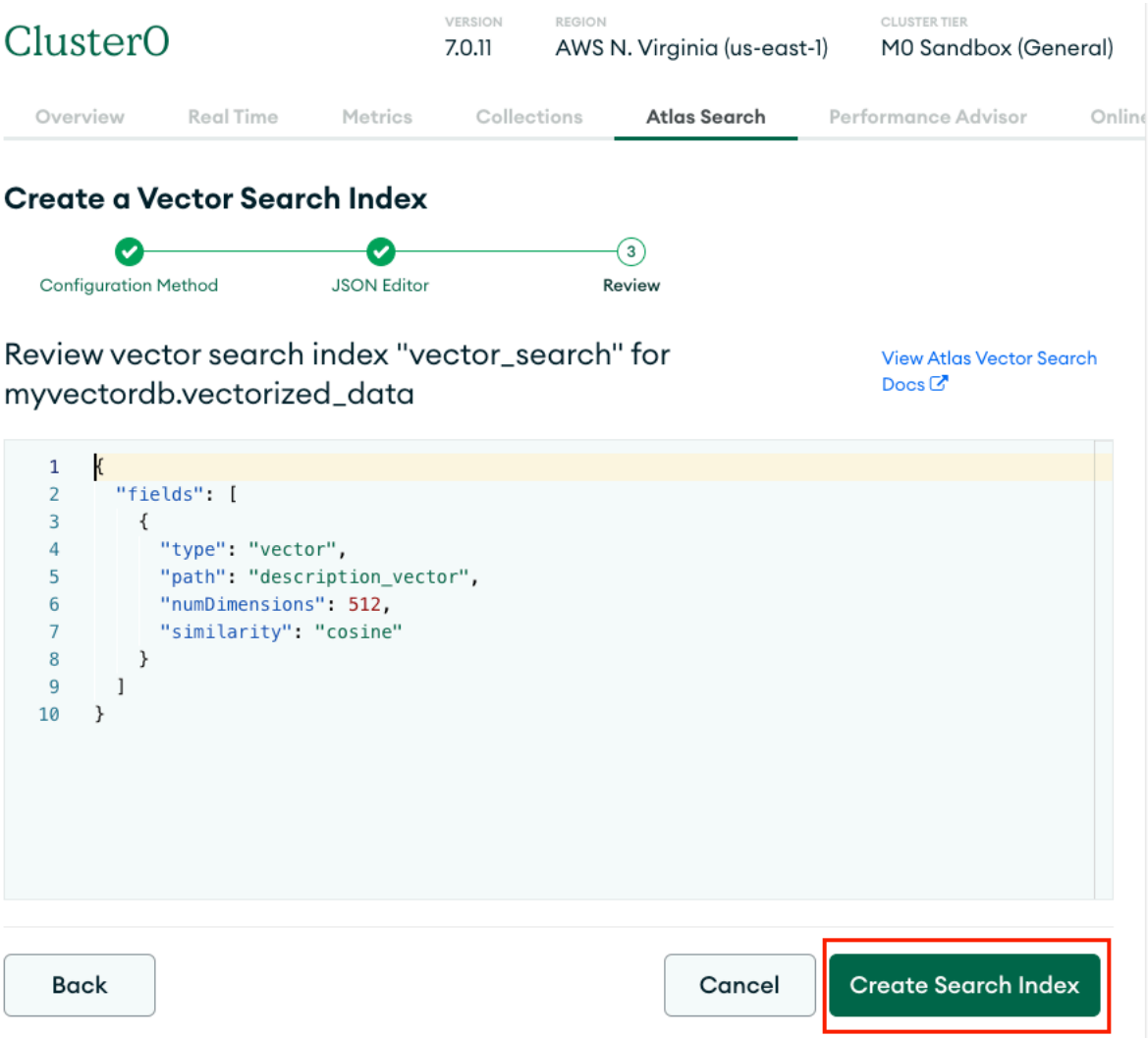
- Pick the database `vectorized_db` as the database.
- Call your index `vector_search`.
- Add the following JSON to the actual index content:

```
{
  "fields": [
    {
      "type": "vector",
      "path": "description_vector",
      "numDimensions": 512,
      "similarity": "cosine"
    }
  ]
}
```

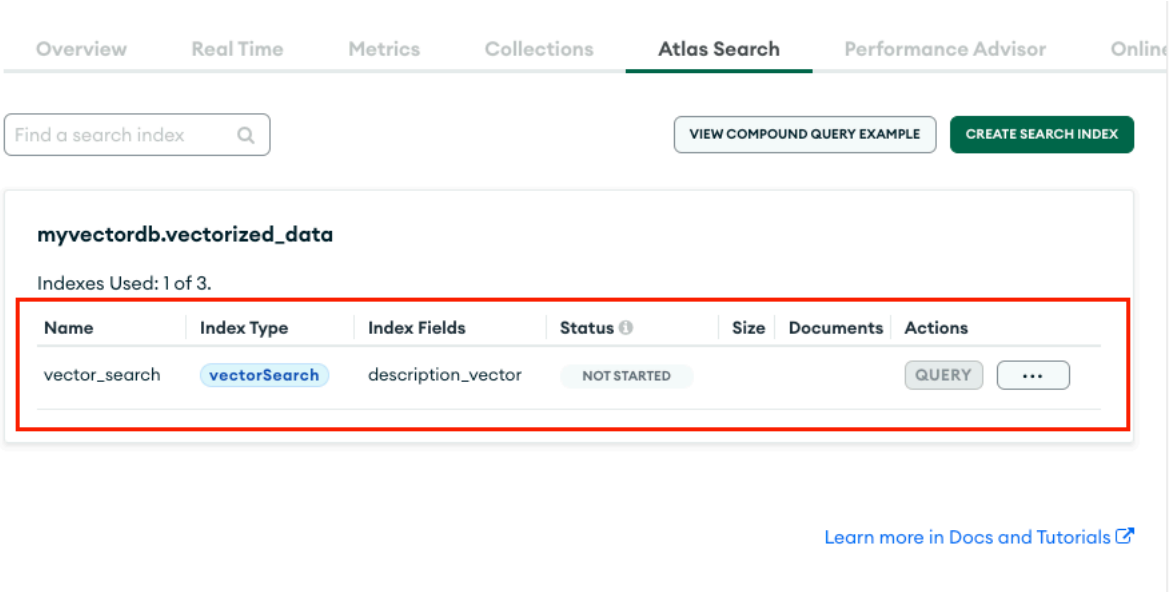
Your final screen should look like this.



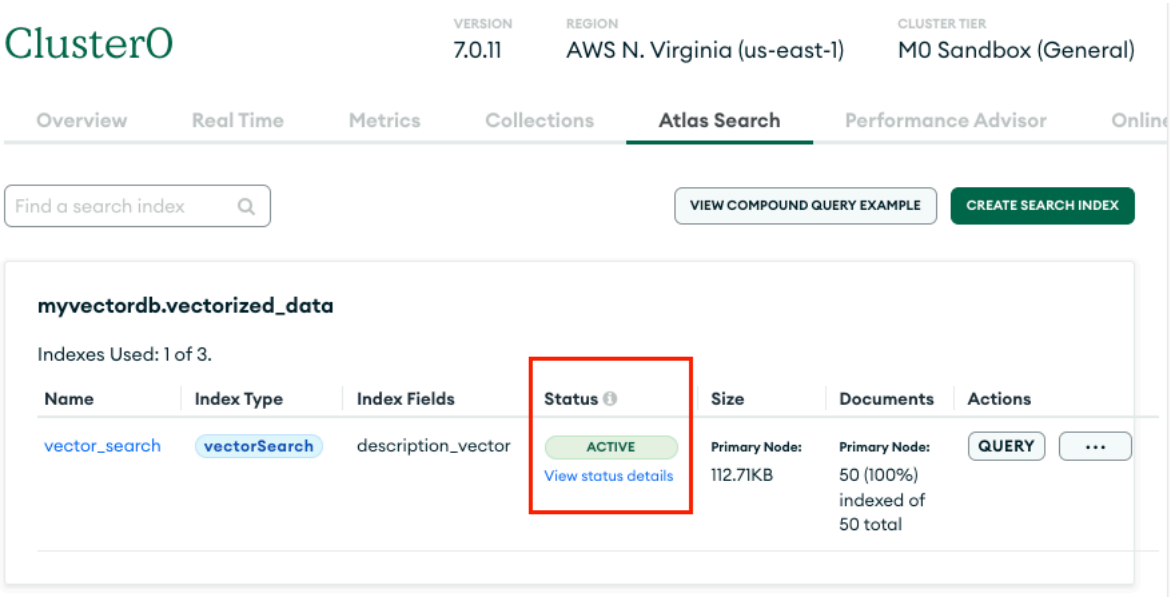
You should see a summary screen. Click on `Create search index` to create the index.



You return to the main page and see your index with a status of `NOT STARTED`.



The status should change to `Active` in a few minutes.



That's all for this step. You successfully created a search index. The next step is to create the service to use this index.

Create a service endpoint

In this section, we will create a service with an endpoint that can take a sentence as input and return the closed news article to that sentence. Let's see this in action using the curl command, which looks for news `with bits and bytes`. The expectation is to get results based on computers or technology.

```
curl -X POST http://localhost:3000/search \
     -H "Content-Type: application/json" \
     -d '{"text": "news with bits and bytes?"}' | jq .
```

Result:


```
 {
   "link": "https://www.huffpost.com/entry/meta-400-million-irish-fine_n_6316c7d0e4b046aa022d0682",
   "headline": "Meta, Parent Company Of Instagram, Fined $400 Million By Irish Regulators",
   "score": 0.6078922152519226
 },
 {
   "link": "https://www.huffpost.com/entry/ap-us-los-angeles-schools-cyberattack_n_63184088e4b046aa022f1bd4",
   "headline": "Cyberattack Prompts Los Angeles School District To Shut Down Its Computer Systems",
   "score": 0.5945149660110474
 },
 {
   "link": "https://www.huffpost.com/entry/unemployment-benefits-recession_n_6317aa40e4b0faa556c1c634",
   "headline": "The Unemployment Insurance System Is Not Ready For The Next Recession",
   "score": 0.5892423391342163
 }
]
```


The results are indeed technology-related, even though the words `bit` or `bytes` were not used in any of the news articles. Note that we also use the tool `jq` to format the output in addition to the 'curl' command.

Let's create this service from scratch now.

1. Create a file called `server.js`.

```
touch server.js
```

2. Copy and paste the following code into the file:

```
const express = require('express');
const { vectorizeText, connectToMongoDB, closeMongoDBConnection, MONGO_COLLECTION } = require('./util');
const app = express();
app.use(express.json());
app.post('/search', async (req, res) => {
    const text = req.body.text;
    if (!text) {
        return res.status(400).send({ error: 'Text is required' });
    }
    try {
        const db = await connectToMongoDB();
        const collection = db.collection(MONGO_COLLECTION);
        const query = await vectorizeText(text);
        const agg = [
            {
                '$vectorSearch': {
                    'index': 'vector_search',
                    'path': 'description_vector',
                    'queryVector': query,
                    'numCandidates': 5,
                    'limit': 5
                }
            },
            {
                '$project': {
                    '_id': 0,
                    'link': 1,
                    'headline': 1,
                    'score': {
                        '$meta': 'vectorSearchScore'
                    }
                }
            },
            {
                '$match': {
                    'score': { '$gt': 0.4 }
                }
            }
        ];
        const results = await collection.aggregate(agg).toArray();
        res.status(200).send(results);
    } catch (error) {
        console.error('Error', error);
        res.status(500).send({ error: 'Internal server error' });
    } finally {
        await closeMongoDBConnection();
    }
});
const PORT = process.env.PORT || 3000;
app.listen(PORT, () => {
    console.log(`Server is running on http://localhost:${PORT}`);
});
```

Let's go through this code line by line to understand what is going on. We first import the required packages:

```
const express = require('express');
const { vectorizeText, connectToMongoDB, closeMongoDBConnection, MONGO_COLLECTION } = require('./util');
```

- we are using `express` to create the server
- the other utilities are being imported just like as did in previous steps from util.js file

Create a POST endpoint

```
app.post('/search', async (req, res) => {
    const text = req.body.text;
    if (!text) {
        return res.status(400).send({ error: 'Text is required' });
    }
```

- we use `app.post()` to create an endpoint at /search URL
- we use `red.body.text` to get the sentence passed in by the client
- we check if the input is empty, and if so, send an error back immediately

Connect to the database and convert the test to a vector

We need to convert the input text to a vector to use the Atlas features to search for similar news items.

```
const db = await connectToMongoDB();
const collection = db.collection(MONGO_COLLECTION);
const query = await vectorizeText(text);
```

- you can see we did not write any new code for this. We are using the utils methods we imported from util.js file

Create aggregation

Next, we will create an aggregation pipeline for a MongoDB collection to perform a vector search and filter the results based on similarity scores.

```
const agg = [
            {
                '$vectorSearch': {
                    'index': 'vector_search',
                    'path': 'description_vector',
                    'queryVector': query,
                    'numCandidates': 5,
                    'limit': 5
                }
            },
            {
                '$project': {
                    '_id': 0,
                    'link': 1,
                    'headline': 1,
                    'score': {
                        '$meta': 'vectorSearchScore'
                    }
                }
            },
            {
                '$match': {
                    'score': { '$gt': 0.4 }
                }
            }
        ];
const results = await collection.aggregate(agg).toArray();
```

- The `$vectorSearch` stage performs the vector search based on the `vector_search` index and the `description_vector` attribute. `numCandidates` is the number of candidates MongoDB should consider.
- The `$project` stage shapes the documents to be returned by the query. The `'score': { '$meta': 'vectorSearchScore' }` includes the similarity score (how similar each result is to the query vector) in the output. A number close to 1 indicates perfect similarity.
- The `$match` stage filters the results based on the similarity score. In our example, only scores greater than 0.4 will be returned.
- Finally, the `collection.aggregate` function runs the whole query and puts the result in the `results` variable.

Run the service

Start the server

The last part of the code stars the app server and listens on port 3000

```
const PORT = process.env.PORT || 3000;
app.listen(PORT, () => {
    console.log(`Server is running on http://localhost:${PORT}`);
});
```

Run the server

You can use the same terminal you used for the other tasks or create a new terminal. Use the `node` command to run the server

```
node server.js
```

You should see an output as follows:

```
➜  mongodb-vector node server.js
Platform node has already been set. You are overwriting the platform with node.
mongodb://root:root@localhost:27017?directConnection=true
Server is running on http://localhost:3000
```

If you execute the curl command shown at the beginning of this lab, you should see a news article related to technology!

Command:

```
curl -X POST http://localhost:3000/search \
     -H "Content-Type: application/json" \
     -d '{"text": "news with bits and bytes?"}'
```

Output:

```
~ curl -X POST http://localhost:3000/search \
     -H "Content-Type: application/json" \
     -d '{"text": "news with bits and bytes?"}'
[{"link":"https://www.huffpost.com/entry/meta-400-million-irish-fine_n_6316c7d0e4b046aa022d0682","headline":"Meta, Parent Company Of Instagram, Fined
```

You can try other examples as well. Here is one more with the text “weather”:

```
➜  ~ curl -X POST http://localhost:3000/search \
     -H "Content-Type: application/json" \
     -d '{"text": "news about the weather"}' | jq .
  % Total    % Received % Xferd  Average Speed   Time    Time     Time  Current
                                 Dload  Upload   Total   Spent    Left  Speed
100  1053  100  1019  100    34   7657    255 --:--:-- --:--:-- --:--:--  8702
[
  {
    "link": "https://www.huffpost.com/entry/ap-us-austin-airport-power-outage_n_6318c91fe4b046aa02302b96",
    "headline": "Power Outage At Austin Airport Leads To Flight Delays, Traffic Jams",
    "score": 0.7094167470932007
  },
  {
    "link": "https://www.huffpost.com/entry/ap-us-austin-airport-power-outage_n_6318c91fe4b046aa02302b96",
    "headline": "Power Outage At Austin Airport Leads To Flight Delays, Traffic Jams",
    "score": 0.7094167470932007
  },
  {
    "link": "https://www.huffpost.com/entry/fairview-fire-california_n_6316e991e4b0536be048168d",
    "headline": "Fast-Moving Fairview Fire Kills At Least 2 In California",
    "score": 0.6908472180366516
  },
  {
    "link": "https://www.huffpost.com/entry/fairview-fire-california_n_6316e991e4b0536be048168d",
    "headline": "Fast-Moving Fairview Fire Kills At Least 2 In California",
    "score": 0.6908472180366516
  },
  {
    "link": "https://www.huffpost.com/entry/jackson-water-pressure-back-to-normal_n_631708e6e4b0ed021deaf884",
    "headline": "Mississippi Governor Says Water Pressure Is Now 'Solid' In Jackson",
    "score": 0.6530711054801941
  }
]
```

Again, we are using the tool `jq` to format the output so it's easier to read.

Congratulations! You successfully created vector embeddings for news articles, created an index on this vector, and finally created an endpoint that can be used to retrieve similar news!

Authors

UL